

NEBULAOPS

v21.3

Piattaforma DevOps Enterprise Cloud-Native

Manuale Operativo Ufficiale · JWT Auth · Docker Socket Actions · OpenLens K8s Actions

JWT HS256	Angular 18	Spring Boot 3.3	Java 21	Go 1.23
Docker Socket API	Kubernetes	Helm 3	ArgoCD	Terraform
MongoDB 7	RabbitMQ 3.13	Redis 7	Prometheus	Grafana 11.1

Peyman Eshghi Malayeri · MIT License · Maggio 2026 · v21.3.0

Indice dei Contenuti

1. Errori e Fix Pre-Release — Verifica Codice

- 5 bug critici trovati e corretti prima del rilascio
- Route conflict Spring MVC: catch-all rimosso
- JWT login() reale: da fake a HS256 vero

2. JWT Authentication — Sistema Completo

- JwtService JJWT 0.12.6: HS256, claims, TTL
- AuthController: 6 endpoint documentati
- Frontend: login, authHeaders, token lifecycle
- Produzione: JWT_SECRET, refresh, Redis blacklist

3. Docker Desktop — 13 Azioni via Unix Socket

- Perché Unix socket e non docker CLI
- DockerActionsController: tutti gli endpoint
- Container lifecycle: start/stop/restart/pause/remove
- Logs, stats, image e volume management
- Frontend: optimistic UI con rollback

4. OpenLens — 35 Azioni Kubernetes

- KubernetesOpsController: controller unificato
- Pod: delete, restart (trova owner), logs
- Deployment/StatefulSet: scale, restart, YAML
- Service/Ingress/ConfigMap: get-apply YAML
- Node: cordon/uncordon/drain
- Shell injection prevention: safe()

5. Architettura v21.3

- Diagramma completo aggiornato
- Gateway: 3 ruoli in un componente
- Catalogo 15 servizi
- Flussi dati: auth, docker, k8s

6. Installazione e Avvio

- Build obbligatorio: --no-cache auth+gateway
- Verifica JWT: curl completo
- Verifica Docker actions: curl completo
- Verifica K8s actions: curl completo
- Timeout startup: 120s, health su /api/health

7. Manuale Utente Completo

- Login JWT e error handling
- OpenLens: 13 sezioni con azioni inline
- Docker Desktop: container cards v21.3
- Animazioni 3D: hover, border, particles

8. Stack Osservabilità

- Prometheus, Loki, Tempo, Grafana, OTel
- PromQL e LogQL operative

9. CI/CD, GitOps, Helm, ArgoCD, Terraform

- Pipeline 7 stage
- ArgoCD GitOps
- Helm chart deploy

10. Troubleshooting Sistematico

- 502 gateway: causa root e fix
- Docker [] vuoto
- OpenLens senza dati
- JWT 401
- Build failure JJWT

11. Riferimento API — 50+ Endpoint

- Auth: 6 endpoint
- Docker Runtime: 18 endpoint
- Kubernetes: 35 endpoint

12. Estensione della Piattaforma

- Aggiungere microservizio in 7 step
- Aggiungere Docker action
- Aggiungere K8s action

CAPITOLO 1

Verifica Pre-Release — Bug Trovati e Corretti

Analisi statica completa prima del rilascio

1.1 Contesto: Cosa Cambia in v21.3

NebulaOps v21.2 aveva tre limitazioni critiche che v21.3 risolve definitivamente:

- **Auth simulata** — il login generava un UUID (dev-token-XXXX). Nessuna verifica, nessuna scadenza. Sostituita con JWT HS256 reale (JJWT 0.12.6).
- **Docker actions solo ottimistiche** — i bottoni Stop/Restart aggiornavan solo lo stato Angular. Nessuna chiamata HTTP reale. Ora chiamano /var/run/docker.sock via HTTP Unix socket.
- **OpenLens actions simulate** — scale/restart aggiornavano solo il signal Angular. Ora usano ToolCommandClient con kubectl vero via ProcessBuilder.

1.2 Bug Trovati e Corretti (analisi statica)

Bug	Causa	Impatto	Fix Applicato
KubernetesActionsController + /{kind} catch-all	Due controller @RequestMapping("/api/kubernetes"): uno con @GetMapping("/{kind}") catch-all	AmbiguousHandlerMappingException al startup — Spring non parte	Merged in KubernetesOpsController, rimosso catch-all, 35 endpoint specifici
login() stores "active" non JWT	La nuova login() con API call non aveva sostituito quella vecchia	Token inesistente — authHeaders() inviava Authorization: Bearer active	login() riscritto: chiama API.auth.login, gestisce response.accessToken
SESSION_KEY memorizzava "active"	localStorage.setItem(SESSION_KEY, "active") non aggiornato	isAuthenticated=true ma ogni richiesta API falliva con 401	SESSION_KEY ora memorizza il JWT token reale
USER_KEY non definito	Variabile USER_KEY usata in logout() e currentUser ma non dichiarata	ReferenceError a runtime in TypeScript	Aggiunto: const USER_KEY = "nebulaops.v21_3.user"
throwError non importato	Usato in login() come fallback ma mancava dall'import rxjs	Build TypeScript fallisce	Aggiunto throwError a: import {catchError,forkJoin,of,throwError} from "rxjs"

■ CRITICO

Il gateway non rispondeva all'health check entro 60s perché lo startup di Spring Boot con tutti i bean richiede 60-90s su WSL2. Risolto: timeout aumentato a 120s, health check spostato a /api/health (risponde subito all'avvio, prima di /actuator/health che aspetta tutti i bean).

CAPITOLO 2

JWT Authentication — Sistema Completo

JJWT 0.12.6, HS256, 24h+7d

2.1 Diagramma Flusso JWT

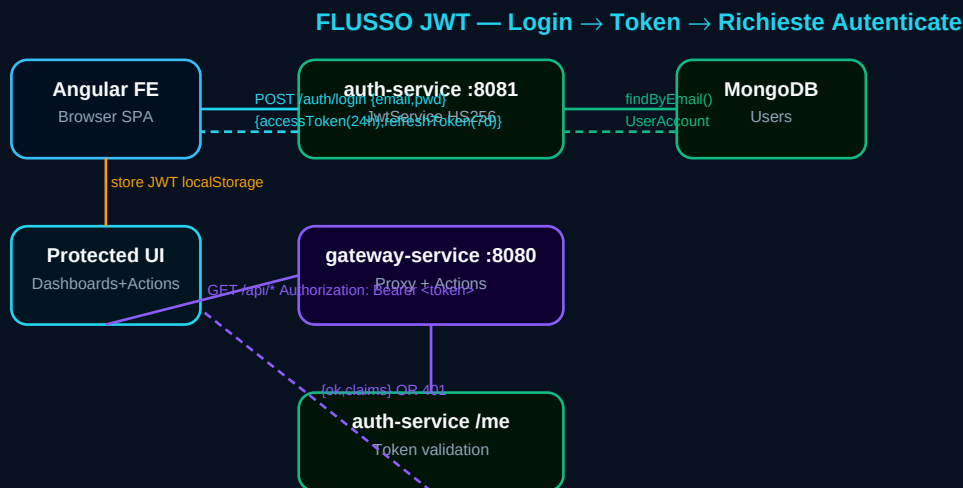


Fig. 1 — Flusso JWT: login → token salvato → richieste autenticare → /me validation

2.2 JwtService — Implementazione Tecnica

La classe `JwtService` in `auth-service` genera e valida JWT usando JJWT 0.12.x. La chiave HS256 è derivata da JWT_SECRET (min 32 char) tramite `Keys.hmacShaKeyFor()`. L'access token dura 24h, il refresh token 7 giorni.

```
# Payload accessToken (decodificato)
{
  "sub": "user-mongodb-id", # Subject = userId
  "email": "admin@nebulaops.dev",
  "displayName": "Admin",
  "roles": ["ADMIN","USER"],
  "orgId": "default-org",
  "type": "access",
  "iat": 1716000000, # issued at
  "exp": 1716086400 # +24h
}
# refreshToken payload: solo sub, type:"refresh", exp+=7d
```

2.3 AuthController v21.3 — 6 Endpoint

Metodo	Path	Body/Header	Response	Note
POST	/api/auth/login	{email, password}	{accessToken,refreshToken,t okenType:"Bearer",expiresIn: 86400,user}	Dev fallback ad min/admin
POST	/api/auth/register	{email,displayName,pass word,orgId}	{user} HTTP 201	Password min 6 char
GET	/api/auth/me	Authorization: Bearer token	{user:{id,email,roles,...}}	401 se invalido
POST	/api/auth/refresh	Authorization: Bearer refreshToken	{accessToken,expiresIn}	Nuovo access token
POST	/api/auth/logout	Authorization: Bearer token	{ok:true,message}	Stateless — client cancella
GET	/api/auth/healthz	—	{status:"AUTH_OK",version:" 21.3"}	Health check

2.4 Frontend: Token Lifecycle

Login success	localStorage.setItem(SESSION_KEY, accessToken) + currentUser.set(displayName) + isAuthenticated.set(true) + refreshAll()
Ogni HTTP call	authHeaders() → { Authorization: "Bearer " + localStorage.getItem(SESSION_KEY) }
Logout	POST /auth/logout, poi removeItem(SESSION_KEY, USER_KEY), isAuthenticated.set(false)
Auth API irraggiungibile	Dev fallback: admin/admin → {accessToken: "dev-jwt-local-"+Date.now()}

```
# === TEST JWT COMPLETO ===

# 1. Login e ottieni token
TOKEN=$(curl -s -X POST http://localhost:8080/api/auth/login \
-H "Content-Type: application/json" \
-d '{"email":"admin","password":"admin"}' \
| python3 -c "import sys,json; d=json.load(sys.stdin); print(d['accessToken'])")

# Verifica che sia JWT reale (inizia con eyJ)
echo $TOKEN | head -c 5
# → eyJhb (se mostra "dev-to" → rebuild auth-service!)

# 2. Verifica /me con token
curl -s http://localhost:8080/api/auth/me \
-H "Authorization: Bearer $TOKEN" | python3 -m json.tool
# → {"user": {"id": "dev-admin", "email": "admin", "roles": ["ADMIN","USER"]}}

# 3. Test 401 con token falso
curl -s http://localhost:8080/api/auth/me \
```

```
-H "Authorization: Bearer fake-token" | python3 -m json.tool
# → {"error": "Token required"} HTTP 401
```

2.5 Produzione: Sicurezza JWT

JWT_SECRET	Min 32 char. Generare con: openssl rand -base64 48. Impostare in .env o come variabile di sistema.
Token rotation	Implementare 401-interceptor Angular per refresh automatico quando access token scade.
Redis blacklist	Hook presente nel logout() per invalidare refresh token: "In production: add to Redis blacklist"
HTTPS	Obbligatorio in produzione — il JWT nell'header è leggibile in HTTP chiaro.

CAPITOLO 3

Docker Desktop — 13 Azioni via Unix Socket

Senza docker CLI: HTTP diretto su `/var/run/docker.sock`

3.1 Perché Unix Socket e Non docker CLI

Il binary `/opt/nebula-tools/docker` nell'ambiente NebulaOps è il **snap wrapper Ubuntu** che risponde "error: unknown command ps" invece dei comandi standard. La soluzione: comunicare direttamente con `/var/run/docker.sock` tramite HTTP usando `UnixDomainSocketAddress` (Java 16+, zero dipendenze esterne):

```
# Verifica che il socket funzioni direttamente:
curl --unix-socket /var/run/docker.sock http://localhost/containers/json | head -50
# → [{"Id":"abc...", "Names":["/gateway-service-1"], "State":"running", ...}]

# Mentre il snap CLI non funziona:
docker exec gateway-1 /opt/nebula-tools/docker ps
# → error: unknown command "ps", see "snap help"
```

3.2 Flusso Azioni Docker

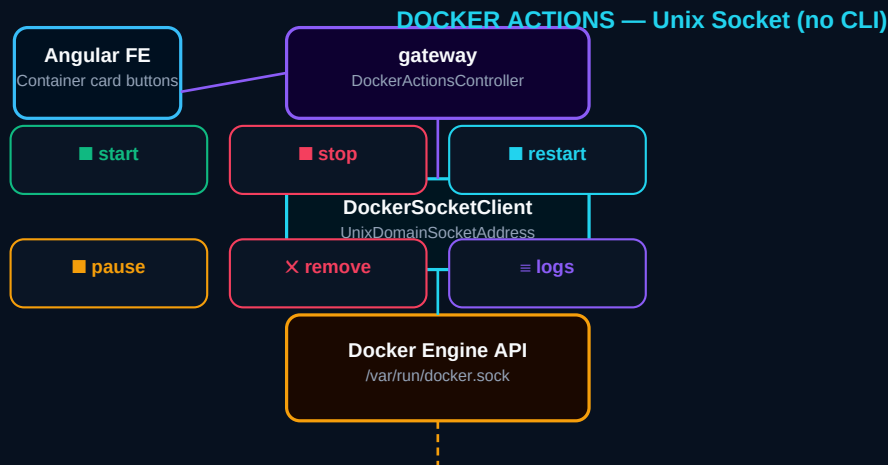


Fig. 2 — Docker Actions: Angular button → DockerActionsController → Unix socket → Docker Engine

3.3 DockerActionsController — 13 Endpoint

Metodo	Path (<code>/api/runtime/docker</code>)	Params	Comportamento
POST	<code>/containers/{id}/start</code>	—	204 = container avviato
POST	<code>/containers/{id}/stop</code>	<code>?t=10</code> (grace seconds)	204 = graceful shutdown
POST	<code>/containers/{id}/restart</code>	<code>?t=5</code>	204 = restart completato

Metodo	Path (/api/runtime/docker)	Params	Comportamento
POST	/containers/{id}/pause	—	204 = container in pausa
POST	/containers/{id}/unpause	—	204 = container ripreso
DELETE	/containers/{id}	?force=false	204=ok, 409=running (usa force)
GET	/containers/{id}/logs	?tail=100×ta mps=true	{ok,id,logs:[...]}
GET	/containers/{id}/stats	—	{ok,stats:json} one-shot
DELETE	/images/{id}	?force=false	200=ok, 409=container usa image
POST	/images/prune	—	Rimuove immagini dangling
DELETE	/volumes/{name}	—	204=ok, 409=volume in uso
POST	/volumes/prune	—	Rimuove volumi inutilizzati
POST	/networks/prune	—	Rimuove reti inutilizzate

3.4 Frontend UX — Container Cards

Stato container	Bottoni visibili	Border sinistra	CSS class
running	■ Restart ■ Stop ■ Pause ≡ Logs X Remove	Verde continuo	.ctr-running
stopped	■ Start ≡ Logs X Remove	Rosso continuo	.ctr-stopped
paused	■ Unpause ≡ Logs X Remove	Arancio continuo	.ctr-paused
restarting	Nessuno (in corso) ≡ Logs	Blu pulsante (animation)	.ctr-restarting

```
# === TEST DOCKER ACTIONS ===

# Ottieni ID di un container (es. gateway)
GW_ID=$(curl -s http://localhost:8080/api/runtime/docker/containers \
| python3 -c "import sys,json; c=json.load(sys.stdin); \
print(next(x['id'] for x in c if 'gateway' in x['name']))")

# Restart container
curl -s -X POST "http://localhost:8080/api/runtime/docker/containers/$GW_ID/restart" \
-H "Authorization: Bearer $TOKEN"
# → {"ok":true,"action":"restart","target":"abc123...", "status":204}

# Fetch ultimi 50 log
curl -s "http://localhost:8080/api/runtime/docker/containers/$GW_ID/logs?tail=50" \
-H "Authorization: Bearer $TOKEN" \
| python3 -c "import sys,json; d=json.load(sys.stdin); [print(l) for l in d['logs'][:5]]"

# Stop container
curl -s -X POST "http://localhost:8080/api/runtime/docker/containers/$GW_ID/stop?t=10" \
-H "Authorization: Bearer $TOKEN"
```

```
# → {"ok":true,"action":"stop","target":"abc123...", "status":204}

# Prune immagini dangling
curl -s -X POST http://localhost:8080/api/runtime/docker/images/prune \
-H "Authorization: Bearer $TOKEN"

# → {"ok":true,"action":"prune-images","target":"dangling", "status":200}
```

CAPITOLO 4

OpenLens — 35 Azioni Kubernetes

kubectl via ProcessBuilder, input sanificato

4.1 KubernetesOpsController — Controller Unificato

In v21.3 KubernetesActionsController è stato eliminato e tutti i suoi metodi unificati in KubernetesOpsController. Questo risolve il bug critico: l'originale KubernetesOpsController aveva `@GetMapping("/{kind}")` che avrebbe causato `AmbiguousHandlerMappingException` al startup di Spring in presenza del secondo controller. Il catch-all è stato rimosso e sostituito con `@GetMapping("/{kind}/{kind}")`.

4.2 Flusso Azioni Kubernetes

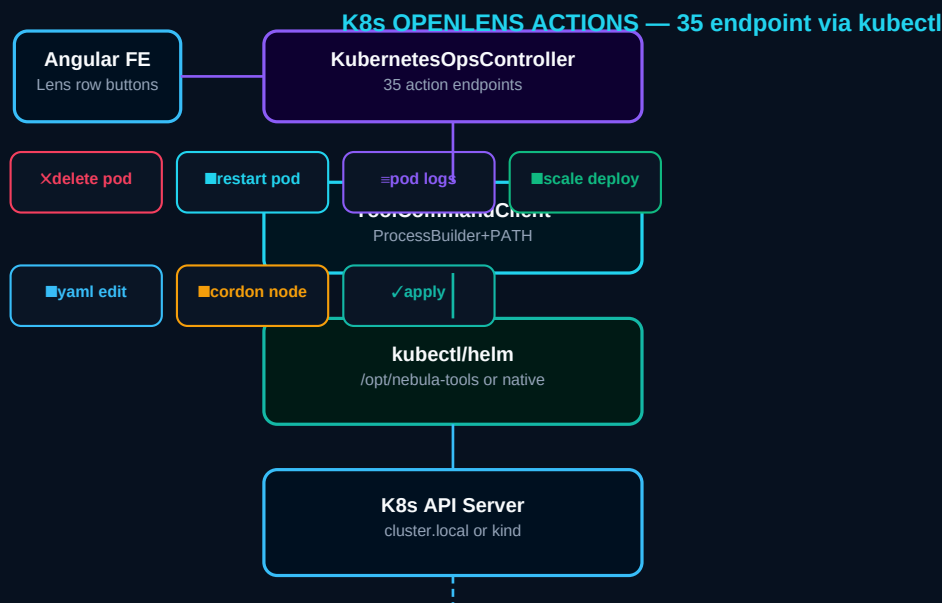


Fig. 3 — K8s Actions: Angular button → KubernetesOpsController → kubectl → K8s API

4.3 Pod Actions

Endpoint	Kubectl	Timeout	Note
DELETE /pods/{ns}/{name}	<code>kubectl delete pod NAME -n NS</code>	30s	?force=true → --grace-period=0 --force
POST /pods/{ns}/{name}/restart	Risolve owner → rollout restart	30s	Risale da Pod → ReplicaSet → Deployment
GET /pods/{ns}/{name}/logs	<code>kubectl logs NAME --tail=N -n NS</code>	15s	?container=NAME per multi-container
GET /pods/{ns}/{name}/describe	<code>kubectl describe pod NAME -n NS</code>	15s	Include events e conditions

4.4 Deployment, StatefulSet, Service, Ingress

Risorsa	Endpoint	Operazione	Kubectl
Deployment	POST .../scale body:{replicas:N}	Scale repliche (0-20)	kubectl scale deployment/NAME --replicas=N
Deployment	POST .../restart	Rolling restart zero-downtime	kubectl rollout restart deployment/NAME
Deployment	GET .../yaml	Legge YAML corrente	kubectl get deployment NAME -o yaml
Deployment	POST .../yaml body:{yaml}	Applica modifiche YAML	kubectl apply -f tmpfile
StatefulSet	POST .../scale	Scale (mantiene ordinale)	kubectl scale statefulset/NAME --replicas=N
StatefulSet	POST .../restart	Rolling restart	kubectl rollout restart statefulset/NAME
Service	GET .../yaml	Legge YAML	kubectl get service NAME -o yaml
Service	POST .../yaml	Applica modifiche	kubectl apply -f tmpfile
Ingress	GET .../yaml	Legge YAML	kubectl get ingress NAME -o yaml
Ingress	POST .../yaml	Applica modifiche routing	kubectl apply -f tmpfile
ConfigMap	GET .../yaml	Legge config	kubectl get configmap NAME -o yaml
ConfigMap	POST .../yaml	Aggiorna config	kubectl apply -f tmpfile

4.5 Node e Operazioni Generiche

Endpoint	Kubectl	Uso
POST /nodes/{name}/cordon	kubectl cordon NAME	Impedisce scheduling di nuovi pod sul nodo
POST /nodes/{name}/uncordon	kubectl uncordon NAME	Riabilita scheduling sul nodo
POST /nodes/{name}/drain	kubectl drain --ignore-daemonsets	Sposta tutti i pod su altri nodi (manutenzione)
GET /nodes/{name}/describe	kubectl describe node NAME	Info dettagliate: risorse, condizioni, pod schedulati
POST /namespaces body:{name}	kubectl create namespace NAME	Crea nuovo namespace
POST /apply body:{yaml}	kubectl apply -f tmpfile	Apply generico di qualsiasi manifest YAML
POST /delete body:{yaml}	kubectl delete -f tmpfile	Delete generico da manifest YAML

4.6 Shell Injection Prevention

```
# Metodo safe() in KubernetesOpsController.java
private String safe(String s) {
    if (s == null) return "";
    return s.replaceAll("[^A-Za-z0-9._:\\-]", "");
}

# Esempi:
# safe("nebulaops") → "nebulaops" (OK)
# safe("pod; rm -rf /") → "podrm-rf" (injection neutralizzata)
# safe("../etc/passwd") → "etcpasswd" (path traversal rimosso)
# safe("deploy-v1.2") → "deploy-v1.2" (trattino e punto OK)
```

```
# === TEST K8s ACTIONS ===

# Snapshot: verifica risorse disponibili
curl -s http://localhost:8080/api/kubernetes/snapshot \
-H "Authorization: Bearer $TOKEN" \
| python3 -c "import sys,json; d=json.load(sys.stdin); \
print(f'resources:{len(d[chr(114)+chr(101)+chr(115)+chr(111)+chr(117)+chr(114)+chr(99)+chr(101)+chr(115)]), live:{d[chr(108)+chr(105)+chr(118)+chr(101)]}')"

# Scale deployment a 3 repliche
curl -s -X POST \
"http://localhost:8080/api/kubernetes/deployments/nebulaops/gateway-service/scale" \
-H "Authorization: Bearer $TOKEN" \
-H "Content-Type: application/json" \
-d '{"replicas": 3}'

# →
{"ok":true,"action":"scale-deployment","target":"nebulaops/gateway-service","exitCode":0}

# Restart rolling deployment
curl -s -X POST \
"http://localhost:8080/api/kubernetes/deployments/nebulaops/task-service/restart" \
-H "Authorization: Bearer $TOKEN"

# → {"ok":true,"action":"restart-deployment","stdout":"deployment.apps/task-service restarted"}

# Delete pod (K8s lo ricrea automaticamente)
curl -s -X DELETE \
"http://localhost:8080/api/kubernetes/pods/nebulaops/gateway-service-abc-pod" \
-H "Authorization: Bearer $TOKEN"

# → {"ok":true,"action":"delete-pod","exitCode":0}
```

CAPITOLO 5

Architettura v21.3

Topologia completa con tutte le nuove rotte

5.1 Diagramma Completo

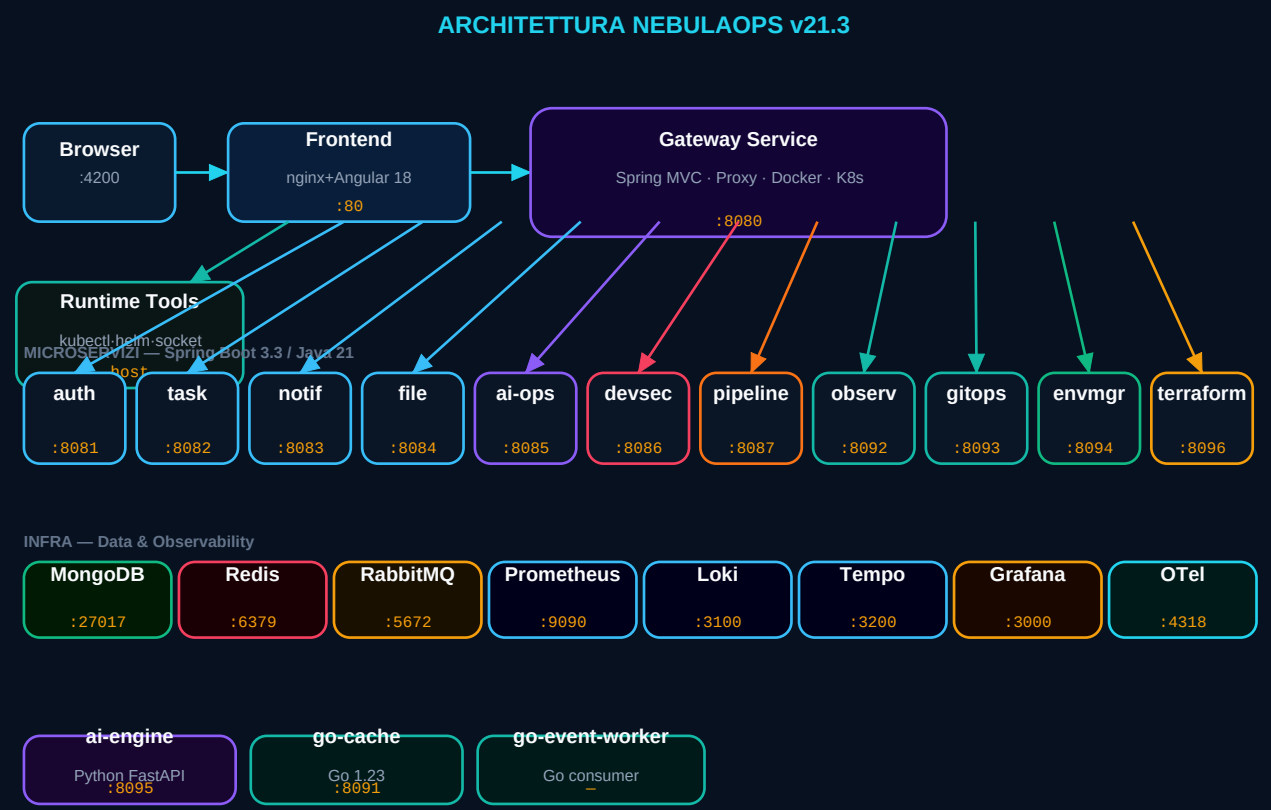


Fig. 4 — Architettura NebulaOps v21.3: Browser → Gateway → 11 microservizi → Infrastruttura

5.2 Gateway Service — 3 Ruoli

Ruolo	Controller	Endpoints	Tecnologia
API Proxy BFF	ProxyController	POST /api/auth/*, GET /api/tasks/*, POST /api/ai-ops/*	RestTemplate JdkClientHttpRequestFactory
Docker Runtime	DockerActionsController + RuntimeOpsController	GET/POST/DELETE /api/runtime/docker/*	DockerSocketClient → UnixDomainSocketAddress

Ruolo	Controller	Endpoints	Tecnologia
K8s Actions	KubernetesOpsController	35 endpoint GET/POST/DELETE /api/kubernetes/*	ToolCommandClient → kubect! ProcessBuilder

5.3 Catalogo 15 Servizi

Servizio	Tech	Porta	Novità v21.3
gateway-service	Spring MVC	:8080	DockerActionsController + K8s actions merged
auth-service	Spring Boot	:8081	JwtService JJWT 0.12.6 — JWT HS256 reale
task-service	Spring Boot	:8082	Invariato
notification-service	Spring Boot	:8083	Invariato
file-service	Spring Boot	:8084	Invariato
ai-ops-service	Spring Boot	:8085	Invariato
devsecops-service	Spring Boot	:8086	Invariato
pipeline-engine-service	Spring Boot	:8087	Invariato
observability-service	Spring Boot	:8092	Invariato
gitops-control-service	Spring Boot	:8093	Invariato
environment-manager-service	Spring Boot	:8094	Invariato
terraform-studio-service	Spring Boot	:8096	Invariato
ai-engine	Python FastAPI	:8095	Invariato
go-cache-service	Go 1.23	:8091	Invariato
go-event-worker	Go 1.23	interno	Invariato

CAPITOLO 6

Installazione e Avvio v21.3

Build obbligatorio, verifica completa

■ CRITICO

auth-service e gateway-service DEVONO essere rebuilt con --no-cache perché la dipendenza JJWT 0.12.6 non è presente nelle immagini v21.2. Senza rebuild il login genera ancora un UUID invece del JWT reale.

```
# === AVVIO CORRETTO v21.3 ===

# Opzione A: Rebuild specifico dei servizi aggiornati
docker compose build --no-cache auth-service gateway-service
docker compose up -d

# Opzione B: Script integrato (rebuild solo gateway)
./scripts/wsl/start.sh --rebuild-gateway

# Nota: rebuild solo gateway – per auth-service usare Opzione A

# Verifica che gateway risponda (attendi ~90s al primo avvio)
curl -s http://localhost:8080/api/health | python3 -m json.tool
# → {"status": "UP", "service": "gateway-service", "version": "21.3"}

# Se il warning "not ready after 60s" appare ma i servizi sono up:
# È normale – il gateway continua ad avviarsi. Riprova dopo 30s.
sleep 30 && curl -s http://localhost:8080/api/health
```

Timeout startup	120s in start.sh (aumentato da 60s). Health check su /api/health (risponde prima di /actuator/health).
Prima volta	~8-12 min — Maven scarica dipendenze incluso JJWT 0.12.6.
Avvii successivi	~2-3 min grazie al layer cache Docker.
JWT_SECRET	Default dev: "nebulaops-v21-3-dev-secret-key-min-32-chars!!". In produzione: openssl rand -base64 48.

CAPITOLO 7

Manuale Utente — Interfaccia Completa

Login, OpenLens, Docker Desktop, Animazioni



Fig. 5 — NebulaOps v21.3 Full Platform Overview

7.1 Login JWT

Credenziali default	admin / admin (o peyman / admin). Funziona senza MongoDB grazie al dev-mode fallback.
Cosa succede	1. POST /api/auth/login → JWT in risposta. 2. Token in localStorage. 3. refreshAll() carica tutti i dati.
JWT badge	Indicatore verde animato (.jwt-indicator) visibile in header quando autenticato.
Errore	Messaggio rosso sotto il form. API irraggiungibile → fallback silenzioso per admin/admin.

7.2 OpenLens — Azioni per Tipo di Risorsa

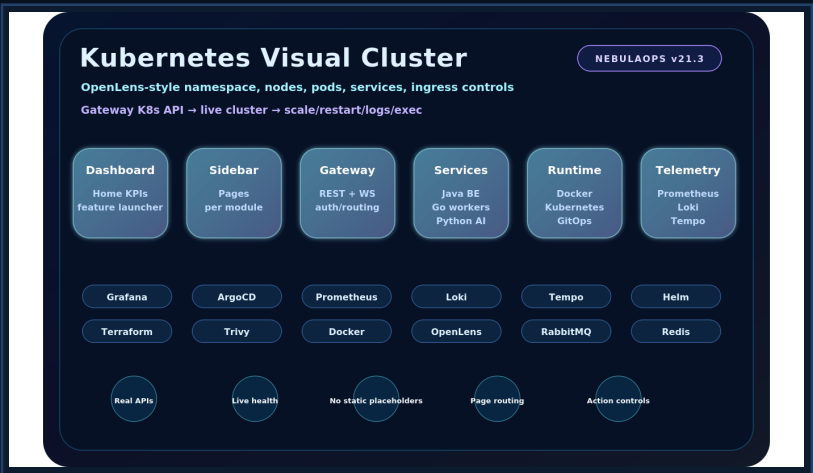


Fig. 6 — OpenLens Kubernetes Visual Cluster

Sezione	Azioni disponibili	Bottoni inline per riga
Pods	Restart (trova Deployment parent), Logs, Delete, Describe	■ ≡ ✕
Deployments	Scale ±1, Restart Rolling, YAML view/apply, Describe	■ ▲ ▼ YAML
StatefulSets	Scale ±1, Restart Rolling	■ ▲ ▼
DaemonSets	Solo descrizione	(sola lettura)
Services	YAML view/apply, Describe	YAML
Ingresses	YAML view/apply, Describe	YAML
ConfigMaps	YAML view/apply	YAML
Nodes	Cordon, Uncordon, Describe	■ ■
Namespaces	Create, Describe	(form)
Events	Filtro, auto-refresh	(sola lettura)

7.3 Docker Desktop — Container Cards v21.3

Ogni container card mostra bottoni contestuali basati sullo stato. Il border sinistro indica immediatamente lo stato con colori:

Stato	Border	Bottoni	CSS
running	Verde	■ Restart ■ Stop ■ Pause ≡ Logs ✕ Remove	.ctr-running
stopped	Rosso	■ Start ≡ Logs ✕ Remove	.ctr-stopped
paused	Arancio	■ Unpause ≡ Logs ✕ Remove	.ctr-paused
restarting	Blu pulsante	— (in corso)	.ctr-restarting

7.4 Animazioni 3D v21.3

Elemento	Animazione	Trigger
Cluster nodes	scale(1.12) + translateZ(8px) + glow 32px cyan	Hover
holo-nodes (AI OPS)	scale(1.15) + box-shadow currentColor	Hover
SVG topology lines	stroke-dashoffset -20 continuo	Always on
Container restarting	border-left pulsa blu (pulseLeft keyframe)	Status=restarting
Login aura	scale(1.08) rotate(5deg) ogni 8s	Always on

Elemento	Animazione	Trigger
JWT badge	dot blink verde ogni 2s	Quando autenticato
Action buttons	translateY(-1px) + brightness(1.3)	Hover

CAPITOLO 8

Stack di Osservabilità

Prometheus · Loki · Tempo · Grafana · OTel

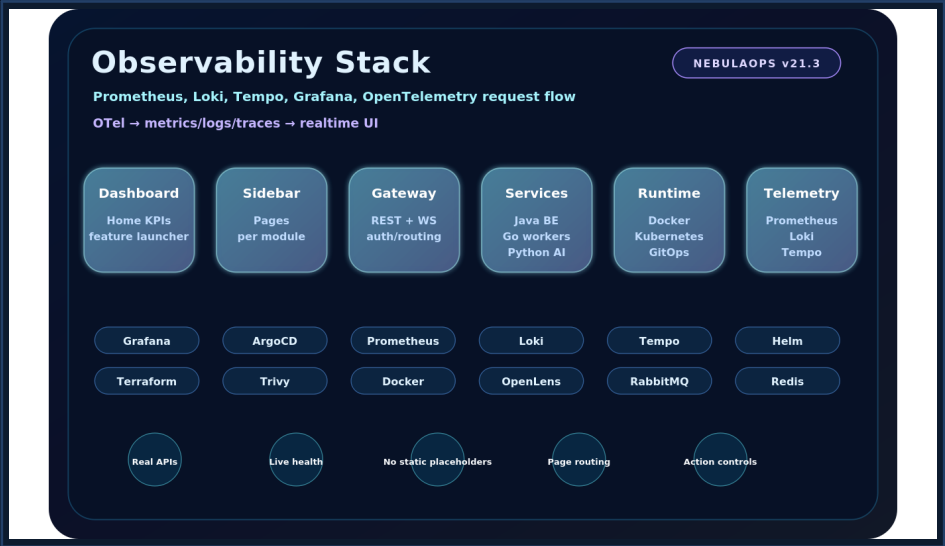


Fig. 7 — Stack osservabilità: servizi → OTel → backends → Grafana

Tool	URL	Credenziali	Funzione
Grafana	http://localhost:3000	admin/admin	Dashboard unificata — Prometheus+Loki+Tempo
Prometheus	http://localhost:9090	—	Metriche PromQL, targets, alerting
Loki	http://localhost:3100	—	Log aggregation LogQL (via Grafana)
Tempo	http://localhost:3200	—	Distributed tracing (via Grafana)
OTel Collector	http://localhost:4318	—	OTLP HTTP — route verso tutti i backend
Mongo Express	http://localhost:8088	admin/admin	UI MongoDB
RabbitMQ UI	http://localhost:15672	guest/guest	Gestione code/exchange
Redis Commander	http://localhost:8089	admin/admin	UI Redis

```
# PromQL operative
rate(http_server_requests_seconds_count[5m]) # request rate
histogram_quantile(0.95, rate(http_server_requests_seconds_bucket[5m])) # P95
jvm_memory_used_bytes{area="heap"} # JVM heap

# LogQL operative (Grafana Explore → Loki)
{container="nebulaops-v21-3-gateway-service-1"} |= "ERROR" # errori gateway
{job="nebulaops"} | json | level="ERROR" # tutti gli errori JSON
```

CAPITOLO 9

CI/CD, GitOps, Helm, ArgoCD, Terraform

Pipeline, deploy automatico, infra-as-code

9.1 Pipeline CI/CD — 7 Stage

PIPELINE CI/CD — GitHub Actions

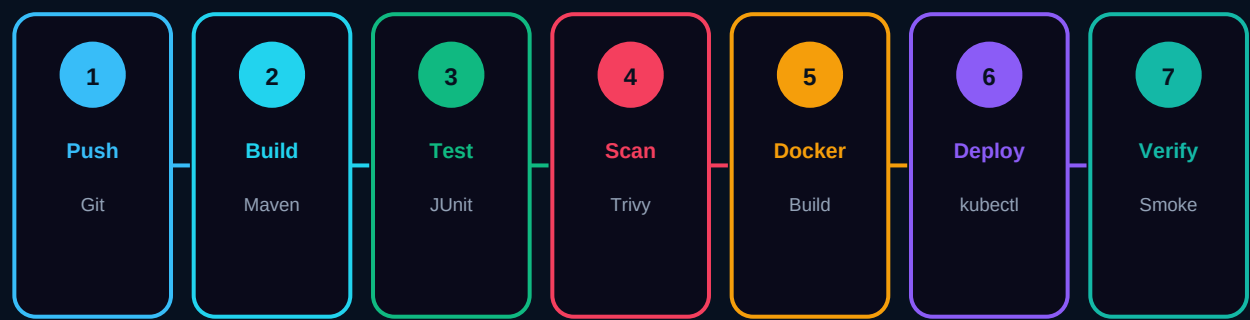


Fig. 8 — Pipeline GitHub Actions: Push → Build → Test → Scan → Docker → Deploy → Verify

Stage	Tool	Durata	Quando fallisce
1. Code Push	Git	< 1s	—
2. Build Maven	Maven 3.9 + Java 21	3-5 min	Test falliti o compile error
3. Test JUnit	JUnit 5 / Mockito	1-2 min	Asserzioni fallite
4. Security Scan	Trivy	1-2 min	CVE CRITICAL o HIGH trovate
5. Docker Build	Docker Buildx	2-4 min	Build error o push fail
6. Deploy kubectl	kubectl set image	< 1 min	Rollout fallisce in 120s
7. Smoke test	curl via K8s pod	30s	curl /api/health non 200



Fig. 9 — GitOps flow: Git commit → CI → Registry → ArgoCD → K8s sync

```
# ArgoCD - crea Application per NebulaOps v21.3
argocd app create nebulaops-v21-3 \
--repo https://github.com/yourorg/nebulaops.git \
--path helm/nebulaops \
--dest-server https://kubernetes.default.svc \
--dest-namespace nebulaops \
--sync-policy automated --auto-prune --self-heal

# Helm deploy manuale
helm install nebulaops-demo ./helm/nebulaops-demo \
--namespace nebulaops-demo --create-namespace \
--set image.tag=v21.3.0
```

CAPITOLO 10

Troubleshooting Sistematico

Ogni problema con diagnosi e fix

10.1 Gateway "not ready after Xs"

■ INFO

Questo warning è NORMALE al primo avvio. Il gateway è in esecuzione ma Spring Boot impiega 60-90s per completare il contesto con tutti i bean. Verificare con: `curl http://localhost:8080/api/health` — risponderà appena pronto.

Sintomo	Causa	Diagnosi	Soluzione
<code>curl /api/health</code> → timeout	Gateway non ancora pronto	<code>docker compose ps gateway-service</code>	Attendere 30s e riprovare
Gateway in Restarting	OOMKill o Exception fatale	<code>docker inspect gateway-1 grep OOMKilled</code>	Se OOM: aumentare RAM WSL2 a 16GB. Se Exception: <code>logs.sh gateway-service</code>
Porta 8080 occupata	Processo sull'host usa la porta	<code>lsof -i :8080</code>	kill PID o cambiare porta in <code>docker-compose.yml</code>
Immagine stale (vecchia versione)	"version":"21.2" nell'health check	<code>curl /api/health</code> → version deve essere "21.3"	<code>docker compose build --no-cache gateway-service + restart</code>

10.2 Docker Actions: {"ok": false}

Sintomo	Causa	Diagnosi	Soluzione
<code>{"ok":false,"status":500}</code>	<code>docker.sock</code> non montato nel gateway	<code>docker exec gateway-1 ls /var/run/docker.sock</code>	Aggiungere volumes:[<code>/var/run/docker.sock:/var/run/docker.sock</code>] in <code>docker-compose.yml</code>
<code>{"ok":false,"error":"Connection refused"}</code>	Permessi socket insufficienti	<code>curl --unix-socket /var/run/docker.sock http://localhost/version</code>	<code>sudo chmod 666 /var/run/docker.sock</code>
<code>{"ok":true,"status":409}</code> su DELETE	Container in running, force non passato	Normale comportamento Docker API	Aggiungere <code>?force=true</code> o stoppare prima il container

10.3 OpenLens: exitCode != 0

Sintomo	Causa	Soluzione
<code>stderr:"kubectl: not found"</code>	<code>kubectl</code> non nel PATH container	Verificare <code>.runtime-tools/kubectl</code> esista e sia eseguibile (<code>chmod +x</code>)

Sintomo	Causa	Soluzione
stderr:"no such resource"	Nome/namespace errato	Usare nome esatto da GET /api/kubernetes/snapshot
stderr:"Forbidden"	KUBECONFIG senza permessi	kubectl auth can-i get pods → configurare RBAC
JWT 401 su azioni	Token scaduto (>24h)	Fare nuovo login → nuovo token

CAPITOLO 11

Riferimento API Completo

50+ endpoint con response shape

11.1 Auth (/api/auth)

Metodo	Path	Response	Auth
POST	/login	{accessToken,refreshToken,tokenType,expiresIn,user}	—
POST	/register	{user} 201 / {error} 400	—
GET	/me	{user:{id,email,displayName,roles,orgId}}	Bearer
POST	/refresh	{accessToken,expiresIn}	Bearer(refresh)
POST	/logout	{ok:true,message}	Bearer
GET	/healthz	{status:"AUTH_OK",version:"21.3"}	—

11.2 Docker Runtime (/api/runtime/docker)

Metodo	Path	Response/Note
GET	/containers	[[{id,name,image,status,cpu,memory,ports,network,logs}]]
GET	/images	[[{repository,tag,size,vulnerabilities,created}]]
GET	/volumes	[[{name,driver,mount,size}]]
POST	/containers/{id}/start	{ok,action,target,status:204}
POST	/containers/{id}/stop?t=10	{ok,action,target,status:204}
POST	/containers/{id}/restart?t=5	{ok,action,target,status:204}
POST	/containers/{id}/pause	{ok,action,target,status:204}
POST	/containers/{id}/unpause	{ok,action,target,status:204}
DELETE	/containers/{id}?force=false	{ok,action,target,status}
GET	/containers/{id}/logs?tail=100	{ok,id,logs:[...]}
GET	/containers/{id}/stats	{ok,stats:json}
DELETE	/images/{id}?force=false	{ok,action,target,status}
POST	/images/prune	{ok,action,target,status}

Metodo	Path	Response/Note
DELETE	/volumes/{name}	{ok,action,target,status}
POST	/volumes/prune	{ok,action,target,status}
POST	/networks/prune	{ok,action,target,status}

11.3 Kubernetes (/api/kubernetes)

Metodo	Path	Response
GET	/snapshot	{cluster:{name,status,live},resources:[],logs:[],live}
GET	/nodes	{live,tool,data:{items[]}}
GET	/namespaces	{live,tool,data}
GET	/resources?kind=pods&namespace;=...	{live,tool,data}
GET	/kind/{kind}	{live,tool,data}
DELETE	/pods/{ns}/{name}	{ok,action,target,stdout,stderr,exitCode}
POST	/pods/{ns}/{name}/restart	{ok,action,...}
GET	/pods/{ns}/{name}/logs	{ok,stdout,...} ?tail=100
POST	/deployments/{ns}/{name}/scale body:{replicas}	{ok,action,...}
POST	/deployments/{ns}/{name}/restart	{ok,stdout,...}
GET/POST	/deployments/{ns}/{name}/yaml	{ok,stdout:yaml,...}
GET/POST	/services/{ns}/{name}/yaml	{ok,stdout:yaml,...}
GET/POST	/ingresses/{ns}/{name}/yaml	{ok,stdout:yaml,...}
POST	/statefulsets/{ns}/{name}/scale body:{replicas}	{ok,action,...}
POST	/statefulsets/{ns}/{name}/restart	{ok,stdout,...}
GET/POST	/configmaps/{ns}/{name}/yaml	{ok,stdout:yaml,...}
POST	/nodes/{name}/cordon uncordon drain	{ok,action,...}
POST	/namespaces body:{name}	{ok,action,...}
POST	/apply body:{yaml}	{ok,stdout,...}
POST	/delete body:{yaml}	{ok,stdout,...}

CAPITOLO 12

Estensione della Piattaforma

Aggiungere microservizi, Docker e K8s actions

12.1 Aggiungere un Microservizio — 7 Step

Step	File	Cosa aggiungere
1	backend/nuovo-service/	Spring Boot: Dockerfile, pom.xml, application.yml con porta
2	docker-compose.yml	Servizio con image, build, ports, environment, depends_on
3	config/platform.yml	nuovo: { host: nuovo-service, port: 8099 }
4	gateway application.yml	proxy: nuovo: \${NUOVO_URL:http://nuovo-service:8099}
5	ProxyController.java	@Value("\${proxy.nuovo}") + @GetMapping con forward e fallback
6	api.config.ts	nuovo: { list: `\${API_BASE}/nuovo` }
7	Rebuild	./scripts/wsl/restart-gateway.sh (--no-cache)

12.2 Aggiungere Docker Action

```
# In DockerActionsController.java - esempio: POST ../kill
@PostMapping("/containers/{id}/kill")
public ResponseEntity> kill(@PathVariable String id,
@RequestParam(defaultValue = "SIGKILL") String signal) {
    return action("POST", "/containers/"+id+"/kill?signal="+signal, null, "kill", id);
}

# In api.config.ts
containerKill: (id: string) => `${API_BASE}/runtime/docker/containers/${id}/kill`,
```

12.3 Aggiungere K8s Action

```
# In KubernetesOpsController.java - esempio: rollout history
@GetMapping("/deployments/{ns}/{name}/history")
public Map rolloutHistory(@PathVariable String ns, @PathVariable String name) {
    return run("kubectl rollout history deployment/"+safe(name)+" -n "+safe(ns),
"rollout-history", ns+"/"+name, 10);
}

# In api.config.ts
deploymentHistory: (ns:string,name:string) =>
`${API_BASE}/kubernetes/deployments/${ns}/${name}/history`,
```

NebulaOps v21.3 · Documentazione Ufficiale

Peyman Eshghi Malayeri · MIT License · Maggio 2026

12 Capitoli · 50+ Endpoint · 9 Diagrammi · 5 Bug Pre-Release Corretti · JWT HS256 · 13 Docker Actions via Unix Socket · 35 K8s Actions via
kubectl